

**CODETANTRA** Home Learn Anywhere 202501090001@mitaoe.ac.in Support Logout

### 3.1.1. Numpy Array Operations

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`.
- The shape of the array using `shape`.
- The total number of elements in the array using `size`.

Assume all input elements are valid integers.

**Input Format:**

- The first line contains two space-separated integers, `r` and `c`, representing the number of rows and the number of columns.
- The next `r` lines contain `c` space-separated integers representing the elements of the array, entered row by row.

**Output Format:**

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

**Note:** Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases

#### numpyarr...

```
1 import numpy as np
2 r,c = map(int,input().split())
3
4 data=[]
5 if r==0 and c==0:
6     arr=np.empty((0,0),dtype=int)
7 else:
8     for _ in range(r):
9         row=list(map(int,input().split()))
10        data.append(row)
11    arr=np.array(data)
```

Average time: 0.024 s (24.08 ms) | Maximum time: 0.126 s (126.00 ms)

2 out of 2 shown test case(s) passed  
10 out of 10 hidden test case(s) passed

Test case 1 (24 ms) [Debug]

| Expected output | Actual output |
|-----------------|---------------|
| 3 4             | 3 4           |
| 1 2 3 4         | 1 2 3 4       |
| 5 6 7 8         | 5 6 7 8       |
| 9 10 11 12      | 9 10 11 12    |
| [[ 1 2 3 4]     | [[ 1 2 3 4]   |
| [ 5 6 7 8]      | [ 5 6 7 8]    |

Terminal | Test cases

< Prev | Reset | Submit | Next >

**CODETANTRA** Home Learn Anywhere 202501090001@mitaoe.ac.in Support Logout

### 3.2.1. Numpy: Matrix Operations

The given code takes two  $3 \times 3$  matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

**Task:**

You are required to compute and display the results of the following matrix operations:

1. Addition (`matrix_a + matrix_b`)
2. Subtraction (`matrix_a - matrix_b`)
3. Element-wise Multiplication (`matrix_a * matrix_b`)
4. Matrix Multiplication (`matrix_a . matrix_b`)
5. Transpose of Matrix A

**Input Format:**

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

**Output Format:**

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Sample Test Cases

#### matrixOp...

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Matrix A:")
5 matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Matrix B:")
8 matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
9
10
11 # Addition
12 print("Addition: (A + B):")
```

Average time: 0.066 s 66.00 ms Maximum time: 0.129 s 129.00 ms

2 out of 2 shown test case(s) passed  
2 out of 2 hidden test case(s) passed

Test case 1 129 ms Debug

| Expected output | Actual output   |
|-----------------|-----------------|
| Enter Matrix A: | Enter Matrix A: |
| 1 2 3           | 1 2 3           |
| 4 5 6           | 4 5 6           |
| 7 8 9           | 7 8 9           |
| Enter Matrix B: | Enter Matrix B: |
| 1 1 1           | 1 1 1           |

Terminal Test cases

Prev Reset Submit Next

**CODETANTRA** Home Learn Anywhere 202501090001@mitaoe.ac.in Support Logout

### 3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- Vertical Stacking:** Stack the two matrices vertically (one below the other).

**Input Format:**

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

**Output Format:**

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

#### stacking.py

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Perform horizontal stacking (hstack)
11 result = np.hstack((arr1, arr2))
12 print("Horizontal stack:")
```

Average time: 0.026 s, 26.25 ms | Maximum time: 0.030 s, 30.00 ms

2 out of 2 shown test case(s) passed  
2 out of 2 hidden test case(s) passed

Test case 1 30 ms Debug

| Expected output | Actual output |
|-----------------|---------------|
| Enter Array1:   | Enter Array1: |
| 1 2 3           | 1 2 3         |
| 4 5 6           | 4 5 6         |
| 7 8 9           | 7 8 9         |
| Enter Array2:   | Enter Array2: |
| 4 5 6           | 4 5 6         |

Terminal Test cases

Prev Reset Submit Next







how to make pdf on - Google Search

codetantra login - Google Search

Course

https://mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c548...

Import favorites

Gmail

YouTube

Maps

Google

**CODETANTRA**

Home

Learn Anywhere

202501090001@mitaoe.ac.in

Support

Logout

**3.2.5. Numpy: Copying and Viewing Arrays**

01:12

The given code takes a list of Integers as Input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the original\_array and assigning it to view\_array.
- Creating a copy of the original\_array and assigning it to copy\_array.

After completing these steps, observe how modifying the view affects the original\_array, while modifying the copy does not.

**Input Format:**

- A single line of space-separated integers.

**Output Format:**

- After modifying the view:

Original array after modifying view: <original\_array>

View array: <view\_array>

After modifying the copy:

Original array after modifying copy: <original\_array>

Copy array: <copy\_array>

Sample Test Cases

copyAnd...

Submit

1

import numpy as np

2

3

inputlist = list(map(int,input().split(" ")))

4

5

# Original array

6

original\_array = np.array(inputlist)

7

8

# Create a view

9

view\_array =original\_array.view()

10

11

# Create a copy

12

copy\_array =original\_array.copy()

Average time

0.007 s

7.50 ms

Maximum time

0.010 s

10.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

10 ms

Debug

Expected output

10 20 30 40 50 60 70 80

Original array after modifying view: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Copy array: [10 80 30 40 50 60 70 80]

Actual output

10 20 30 40 50 60 70 80

Original array after modifying view: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Copy array: [10 80 30 40 50 60 70 80]

Terminal

Test cases

Prev

Reset

Submit

Next

Type here to search

10:09 PM

5/5/2026

**CODETANTRA** Home Learn Anywhere 202501090001@mitaoe.ac.in Support Logout

### 3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, array1, as space-separated integers as input from the user. Additionally, it takes the following inputs:

- search\_value: The value to search for in the array.
- count\_value: The value to count its occurrences in the array.
- broadcast\_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- Searching:** Find the indices where search\_value appears in array1 and print these indices.
- Counting:** Count how many times count\_value appears in array1 and print the count.
- Broadcasting:** Add broadcast\_value to each element of array1 using broadcasting, and print the resulting array.
- Sorting:** Sort array1 in ascending order and print the sorted array.

**Input Format:**

- A single line containing space-separated integers representing array1.
- An integer search\_value represents the value to search for in the array.
- An integer count\_value represents the value to count in the array.
- An integer broadcast\_value represents the value to add to each element of the array.

**Output Format:**

- The indices where search\_value occurs in array1.
- The count of occurrences of count\_value in array1.
- The array after adding the broadcast\_value to each element.
- The sorted array.

arrayOpe...

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split()))))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
12 indices = np.where(array1 == search_value)[0]
```

Average time: 0.019 s, 18.75 ms | Maximum time: 0.025 s, 25.00 ms | 2 out of 2 shown test case(s) passed | 2 out of 2 hidden test case(s) passed

Test case 1 25 ms Debug

|                    |                    |
|--------------------|--------------------|
| Expected output    | Actual output      |
| 1 1 1 2 2 2        | 1 1 1 2 2 2        |
| Value to search: 1 | Value to search: 1 |
| Value to count: 2  | Value to count: 2  |
| Value to add: 2    | Value to add: 2    |
| [0 1 2]            | [0 1 2]            |
| 3                  | 3                  |

Terminal Test cases

how to make pdf on - Google Search

codetantra login - Google Search

Course

https://mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c548...

Import favorites

Gmail

YouTube

Maps

Google

DE TANTRA

Home

Learn Anywhere

202501090001@mitaoe.ac.in

Support

Logout

3.2.7. Student Data Analysis and Operations

17/13

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.

Operatio...

Submit

```
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4
5 # 1. Print all student details
6 print("All student Details:\n",a)
7
8 # 2. print total students
9
10 print("Total Students:",a.shape[0])
11
12 # 3. Print all student Roll numbers
```

Average time

0.028 s

28.00 ms

Maximum time

0.028 s

28.00 ms

1 out of 1 shown test case(s) passed

Test case 1

28 ms

Debug

Expected output

Actual output

All student Details:

[[301, 67, 77, 88.]

[[301, 67, 77, 88.]

[[302, 78, 88, 77.]

[[302, 78, 88, 77.]

[[303, 45, 56, 80.]

[[303, 45, 56, 80.]

[[304, 88, 98, 45.]

[[304, 88, 98, 45.]

[[305, 78, 88, 99.]

[[305, 78, 88, 99.]

Terminal

Test cases

Type here to search

10-10 PM

5/5/2026